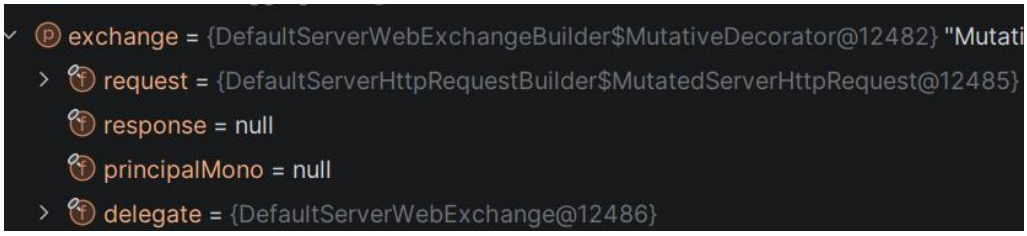
Spring API Gateway Filters

- **GlobalFilter**
  - You can implement a global filter by implementing the interface **GlobalFilter**
  - After that, you need to write the implementation of the method
    Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain chain)
    - This GatewayFilterChain is just to transfer the call to next filter.
    - ServerWebExchange contains all the details like *request, response, attributes* ..etc.
  - You don't need to append this to any specific route, this filter will be attached to every request that goes through the spring cloud api gateway server.
  - Make sure to make the custom global filter as a **Component**.

```java
@Slf4j  new *
@Component
public class GlobalLoggingFilter implements GlobalFilter {

    @Override  new *
    public Mono<Void> filter( @NotNull ServerWebExchange exchange,  @NotNull GatewayFilterChain chain) {

        log.info("request URI : {}", exchange.getRequest().getURI());
        Mono<Void> check = chain.filter(exchange);
        System.out.println(check);
        String responseStatus = exchange.getResponse().getStatusCode().toString();
        log.info("Status : {}", responseStatus);

        return check;
    }
}
```

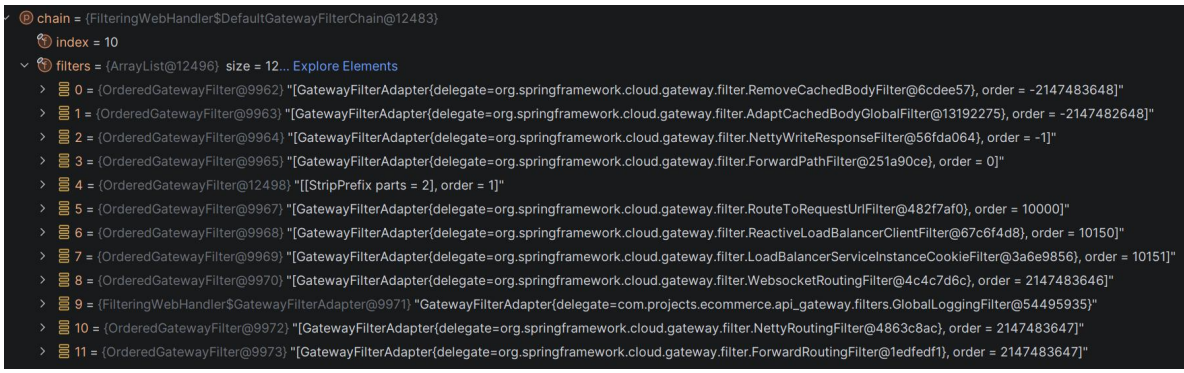
    - Its WRONG ❌
    - **chain.filter(exchange)** that is to execute the next filters.
    - ~~The **GatewayFilterChain** contains the details about *request*, it contains *response* as well but that is ***null*** initially, only after the request is executed response will be there.~~
      ~~Again, the response will not be there inside that same GatewayFilterChain object.~~



    You can see, *response* is null currently.

And, this *filter* method's return type is **Mono** so only after it is subscribed the *response* will be present.

- In **chain** you can see all the filters.



  - At index 9, you can see our GlobalLoggingFilter.

- The correct way to access the **response** is the below

```
@Override  new *
public Mono<Void> filter( @NotNull ServerWebExchange exchange,  @NotNull GatewayFilterChain chain) {

    log.info("request URI : {}", exchange.getRequest().getURI());
    return chain.filter(exchange)
            .then( other: Mono.fromRunnable( runnable: () -> {
                log.info("response status: {}", exchange.getResponse().getStatusCode());
            }));
}
```

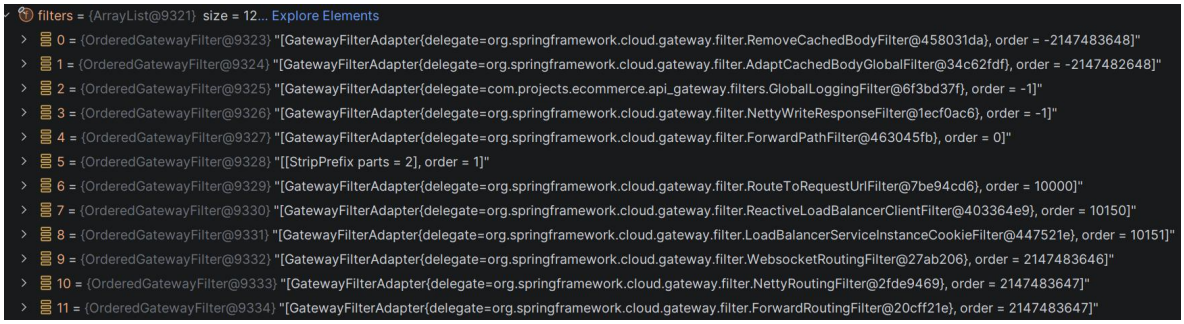
  - Here, we are logging the response only after the Mono (returned object) is subscribed.

    Means, that ***then*** method is executed only after the Mono is subscribed.

- Now, if you want to set the order (here it is coming at index 9 as we saw earlier), then you can implement the **Order** interface.

```
@Slf4j  new *
@Component
public class GlobalLoggingFilter implements GlobalFilter, Ordered {

    @Override  new *
    public Mono<Void> filter( @NotNull ServerWebExchange exchange,  @NotNull GatewayFilterChain chain) {...}

    @Override  new *
    public int getOrder() {
        return -1;
    }
}
```

  - Lowest value : highest priority

- 

    - If you see now, its there at index 2
    - The last values are orders, there are some filters having **negative infinity** ordered so they are at *index 0* and *1*.
  - I wrote 2 custom global filters, all same, the execution order is like

```
ers.GlobalLoggingFilter       : request URI : http://localhost:8080/api/v1/inventory/products
ers.GlobalLoggingFilter2      : request-2 URI : http://localhost:8080/api/v1/inventory/products
ers.GlobalLoggingFilter2      : response-2 status: 200 OK
ers.GlobalLoggingFilter       : response status: 200 OK
```

    - Before ***then***, first to last
    - After ***then***, last to first
  - As you can see, you are passing **exchange** to the **chain.filter()** method, so here the *exchange* object will be properly updated and you can access the response after that.

➢ **GatewayFilter (*Default*)**

  - These filters can be added to specific routes.
  - In simple terms, *Gateway Filters are route-specific filters.*
  -

```
Client
  ↓
(GlobalFilter)
  ↓
(GatewayFilter - only if route matches)
  ↓
Service
```

  - There are built-in gateway filters there like AddRequestHeader, AddRequestHeadersIfNotPresent, AddRequestParameter, …etc

- I added the following headers

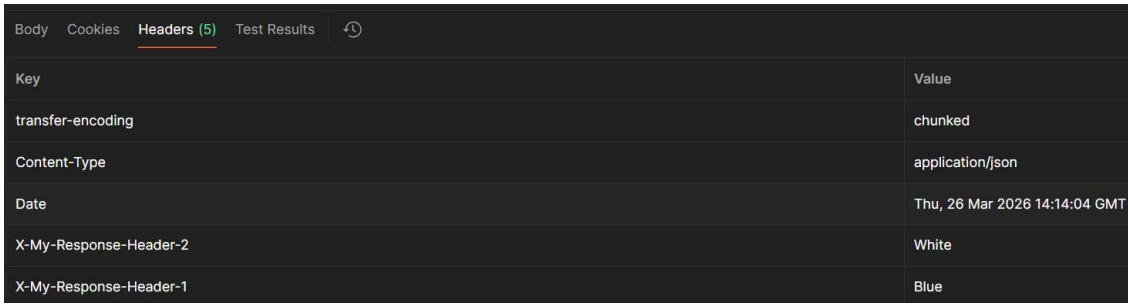
```yaml
- id: inventory-service
  uri: lb://INVENTORY-SERVICE
  predicates:
    - Path=/api/v1/inventory/**
  filters:
    - StripPrefix=2
    - AddRequestHeader=X-My-Request-Header, My-Header-Value
    - AddResponseHeader=X-My-Response-Header-1, Blue
    - AddResponseHeader=X-My-Response-Header-2, White, false
```

  - Request header will be appended to request and can be accessed from the specific services.

```java
@GetMapping  Alok Ranjan Joshi *
public ResponseEntity<List<ProductDto>> getAllInventories( @NotNull HttpServletRequest request) {
    System.out.println(request.getHeader( s: "X-My-Request-Header"));
```

```
Hibernate: select p1_0.id,p1_0.price,p1_0.stock,p1_0.title from product p1_0
My-Header-Value
```

  - Response headers will be attached to the response object got from the service and can be accessed from the client side.



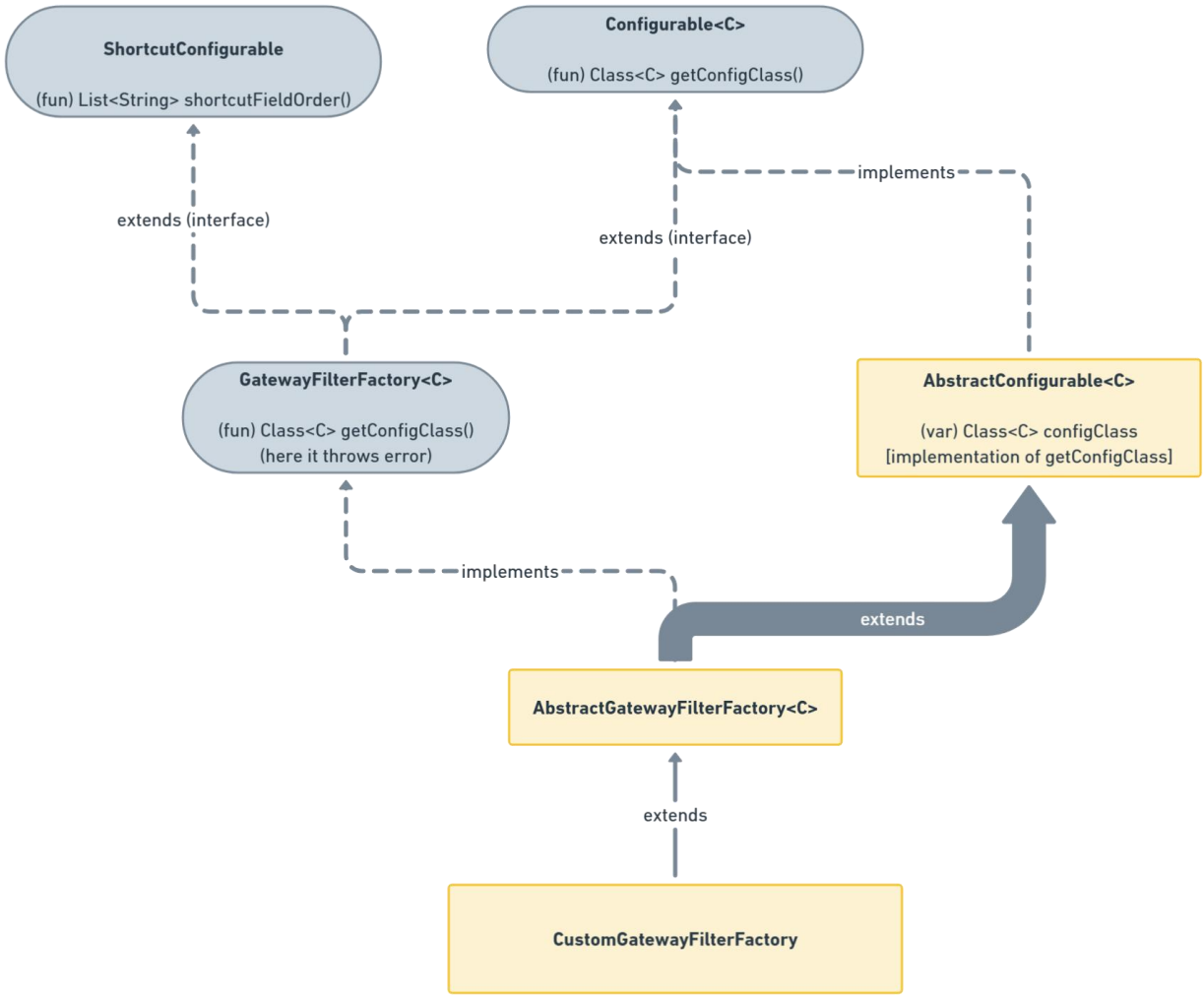
➢ GatewayFilter (*Custom*)

- To create custom gateway filters, you have to implement the abstract class **AbstractGatewayFilterFactory**.
- There is a method **getConfigClass()** in both *GatewayFilterFactory* and *Configurable*, but in when same method is written in 2 places, Java always select the method present in class instead of interface.
- You need to create a *static class* in your custom gateway filter factory class and pass that as the generic, it is required and will be helpful if you want to pass any argument to the gateway filter from yaml file.

```
public abstract class AbstractGatewayFilterFactory<C> extends AbstractConfigurable<C> in
    private ApplicationEventPublisher publisher;

    @Contract(pure = true)
    public AbstractGatewayFilterFactory() { super( configClass: Object.class); }

    @Contract(pure = true)
    public AbstractGatewayFilterFactory(Class<C> configClass) { super(configClass); }
```

```
public abstract class AbstractConfigurable<C> implements Configurable<C>
    private Class<C> configClass;

    @Contract(pure = true)
    protected AbstractConfigurable(Class<C> configClass) {
        this.configClass = configClass;
    }

    public Class<C> getConfigClass() { return this.configClass; }
```

- You need to pass the object of type **Class** i.e. <your config class>.class



- You need to implement the **GatewayFilter apply(Config config)** method

- *GatewayFilter* is nothing but a functional interface that contains a *filter* method which contains **exchange** and **chain** just like the GlobalFilter case.

```
public interface GatewayFilter extends ShortcutConfigurable {  2 usages  36 imple
    String NAME_KEY = "name";
    String VALUE_KEY = "value";

    Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain chain);
}
```

- Also, *make sure to make the custom gateway filter factory class a* ***component***.

```
@Slf4j
@Component
public class CustomGatewayFilterFactory extends AbstractGatewayFilterFactory<CustomGatewayFilterFactory.Config> {

    public CustomGatewayFilterFactory() {  no usages
        super( configClass: CustomGatewayFilterFactory.Config.class);
    }

    @Override
    public GatewayFilter apply( @NotNull Config config) {
        log.info("config variables: {}, {}", config.getMyVar1(), config.getMyVar2());
        return (exchange, chain) -> {
            log.info("request: {}", exchange.getRequest().getURI());
            return chain.filter(exchange)
                    .then( other: Mono.fromRunnable( runnable: () -> {
                        log.info("response status from 'CustomGatewayFilterFactory': {}", exchange.getResponse().getStatusCode());
                    }));
        };
    }

    @Getter @Setter  3 usages
    public static class Config {
        private String myVar1;
        private String myVar2;

        @Contract(pure = true)
        public Config(String myVar1, String myVar2) {  no usages
            this.myVar1 = myVar1;
        }
    }
```

- Because GatewayFilter is a functional interface, so you can directly use lambda syntax for that.

- If your custom gateway filter factory class has suffix as **GatewayFilterFactory** (in my case **CustomGatewayFilterFactory**) then you must directly write **Custom** in that case in the application.properties or yaml file.

- You can mention this filter in application file like the below passing the arguments

```
- id: inventory-service
  uri: lb://INVENTORY-SERVICE
  predicates:
    - Path=/api/v1/inventory/**
  filters:
    - StripPrefix=2
    - name: Custom
      args:
        myVar1: 'var1'
        myVar2: 'var2'
```

- If you want to pass the argument in a single line, then you need to write the implementation of the method **shortcutFieldOrder()** (which is present in ShortcutConfigurable interface)
  -

```
@Override  no usages
public List<String> shortcutFieldOrder() {
    return List.of("myVar1", "myVar2");
}
```

  - myVar1 and myVar2 are the variable names, and when you pass the argument the order should be same.
    - Means, first will be assigned to myVar1, 2nd will be to myVar2
  -

```
filters:
  - StripPrefix=2
  - Custom=var1, var2
```

```
config variables: myVar1 = var1, myVar2 = var2
```

  - Now, if I change the order

```
public List<String> shortcutFieldOrder() {
    return List.of("myVar2", "myVar1");
}
```

```
config variables: myVar1 = var2, myVar2 = var1
```

- You can use the api gateway to authenticate the user as well like jwt token authentication and all, if user is authenticated then trigger the request to micro services otherwise not. A small example of that

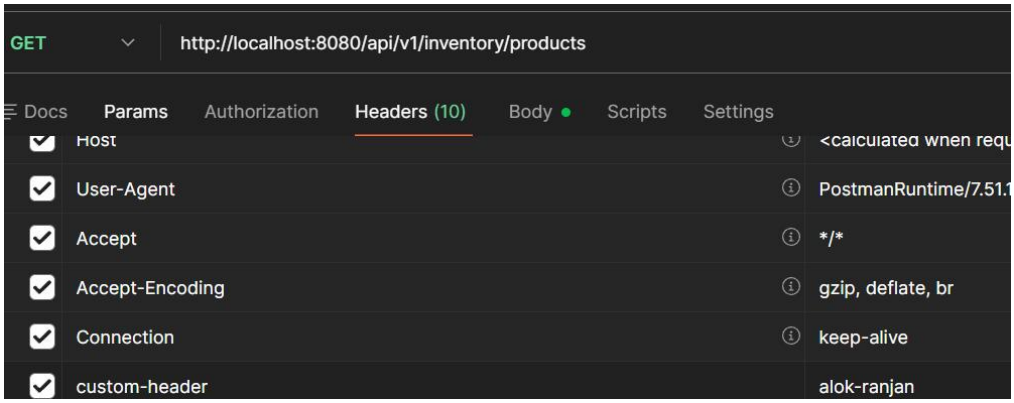
```
@Getter @Setter  3 usages
public static class Config {
    private String myVar1;
    private String myVar2;
    private String headerVal;

    @Contract(pure = true)
    public Config(String myVar1, String myVar2, String headerVal) {
        this.myVar1 = myVar1;
        this.myVar2 = myVar2;
        this.headerVal = headerVal;
    }
}

@Override  no usages
public List<String> shortcutFieldOrder() {
    return List.of( e1: "headerVal");
}
```

```
filters:
  - StripPrefix=2
  - Custom=alok-ranjan
```

  - Now only *headerVal* variable will be assigned, nmyVar1 and myVar2 will be null because I removed those from the shortcutFieldOrder() method.
  - In the long format of application, you can assign the values.

```
- name: Custom
  args:
    myVar1: 'var1'
    myVar2: 'var2'
```
(like this)

  - My concept here is, in the request (from postman), I'll give a header having key "custom-header", if its value is "alok-ranjan" then it'll forward to services otherwise not.
  -




```
@Override
public GatewayFilter apply(Config config) {
    return (exchange, chain) -> {
        log.info("request from 'CustomGatewayFilterFactory: {}", exchange.getRequest().getURI());

        String configHeader = config.getHeaderVal();
        String requestHeader = Objects.requireNonNull( obj: exchange.getRequest().getHeaders().get("custom-header")).getFirst();
        log.info("config headerVal: {}", configHeader);
        log.info("request header val: {}", requestHeader);

        if(!Objects.equals( a: configHeader,  b: requestHeader)) {
            exchange.getResponse().setStatusCode(HttpStatus.BAD_REQUEST);
            return exchange.getResponse().setComplete();
        }
        return chain.filter(exchange)
                .then( other: Mono.fromRunnable( runnable: () -> {
                    log.info("response status from 'CustomGatewayFilterFactory': {}", exchange.getResponse().getStatusCode());
                }));
    };
}
```

- Only in case of correct header, the request will be forwarded otherwise not.

- F
- F
- F
- F
- F
- F